

Semantic Segmentation with Dilated Networks for Autonomous Driving

Mohammadamin Alibakhshi
Deep Learning class, 5743, Professor Hagan
Spring 2026

1 Introduction

Semantic segmentation is the task of assigning a class label to every pixel in an image. In autonomous driving, this is important because the vehicle must understand where the road, sidewalk, vehicles, pedestrians, signs, and other scene parts are located. This is different from image classification, where one label is predicted for the whole image, and it is also different from object detection, where boxes are placed around objects. Semantic segmentation gives a dense prediction, so it is more directly connected to driving scene understanding.

The motivation of this project came from the tradeoff between accuracy and computation. Standard convolutional networks use pooling and strided operations to reduce feature-map size. This is useful for computation, but it can remove spatial detail that is needed for pixel-level prediction. Dilated convolution, also called atrous convolution, increases the receptive field by inserting gaps between kernel points. In this way, the model can see a larger context without adding more weights to the convolution kernel. This means using a larger kernel size with the same computational cost.

The main project question was:

Does dilation improve semantic segmentation performance across different architectures, and what is the cost in runtime?

The project started from the proposal idea of comparing a basic CNN and a dilated CNN. After the baseline was working, the experiment was expanded to VGG16, SegNet-style, and LRASPP models, so the effect of dilation could be tested more broadly.

2 Original proposal direction

The proposal defined the project as semantic segmentation with dilated CNNs for autonomous driving. It planned to use Cityscapes images and fine annotations, build a simple CNN baseline, modify the model with dilated convolutions, and evaluate the result with mIoU and visual prediction examples. The final implementation followed this direction but went further by adding six more models.

The expected challenge from the proposal was that segmentation is more expensive than classification because every pixel must be labeled. This became clear in the implementation because Cityscapes images have resolution $3 \times 1024 \times 2048$, and the batch size had to be kept small for larger models. Although I do not have direct evidence of classification vs. semantic segmentation, training these 8 semantic segmentation networks, four families with and without dilation, took a total of 24 hours. Another expected challenge was that a simple model may miss small objects and boundaries. The visual predictions not only back this up, but we also see a lot of misclassification in the form of noise in the final images, which I didn't expect from a simple CNN, see [Figure 4](#).

3 Dataset

The dataset used in this project is Cityscapes [1]. The code uses TorchVision's Cityscapes dataset class. The predefined Cityscapes fine split contains 2,975 training images and 500 validation images. The input images are left-camera from "leftImg8bit", and the labels are semantic fine annotations from "gtFine".

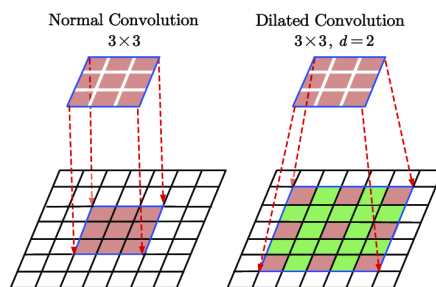


Figure 1: Dilated convolution increases the receptive field without increasing the number of kernel weights. This idea was the main architectural change studied in the project.

4 Label mapping and ignore label

The current code does not train directly on raw Cityscapes label ids. By default, Cityscapes has over 30 finely annotated classes, but we only trained on 19 of them. Regarding implementation, "dataloader.py" creates a train id lookup table from "torchvision.datasets.Cityscapes.classes". The model output has 19 channels, one for each valid Cityscapes training class. All other labels are mapped to 255 and ignored. The ignored label is "255". This is cleaner than forcing unknown or void categories into a normal class, because those pixels do not represent reliable supervision for one of the 19 classes.

Train ID	Class	Train ID	Class
0	road	10	sky
1	sidewalk	11	person
2	building	12	rider
3	wall	13	car
4	fence	14	truck
5	pole	15	bus
6	traffic light	16	train
7	traffic sign	17	motorcycle
8	vegetation	18	bicycle
9	terrain	255	ignored pixels

Training uses cross entropy as the loss function with the same ignore policy "CrossEntropyLoss(ignore index=255)." The mIoU function also first removes ignored pixels. Hence, IoU is computed for classes 0 through 18. Therefore, the loss and the evaluation metric are consistent with each other.

5 Models' specifications

Eight models were trained. Four are baseline models and four are dilated versions. The goal was not to reproduce a state-of-the-art Cityscapes benchmark but to compare the effect of dilation inside an isolated training pipeline.

Model	Implementation summary	Batch size
Simple CNN	Custom encoder-decoder with three convolution blocks, two max-pooling layers, two transposed convolutions, and a final 19-channel classifier.	6
Dilated Simple CNN	Same structure as Simple CNN, but the three encoder convolutions use dilation 4 and padding 4.	6
VGG16	TorchVision vgg16 feature layers up to layer 33; the decoder is manually added and uses three transposed convolutions and two final convolutions.	2
VGG16 Dilated	Same VGG16-style encoder-decoder, but each 3 by 3 encoder convolution uses dilation 2 and padding 2 to keep the output size consistent.	2
SegNet-style	Manual SegNet-inspired encoder-decoder with skip concatenations. The middle block uses dilation 2.	3
SegNet Dilated	Same SegNet-style structure, but most 3 by 3 convolution layers use dilation 4 and padding 4.	3
LRASPP[2]	TorchVision LRASPP with a MobileNetV3-Large backbone. The code uses no pretrained weights and sets the output to 19 classes.	5
LRASPP Dilated	Same LRASPP model, but non- 1×1 backbone convolutions are changed to dilation 2.	5

6 Training and evaluation pipeline

All models use the shared pipeline in "helper.py". The main steps are:

1. Load Cityscapes training and validation splits.
2. Convert images to tensors and map raw labels to train IDs.
3. Train with CrossEntropyLoss function with ignore index=255.
4. Optimize with Adam using learning rate 10^{-4} . (This has remained consistent across all eight models.)
5. Train for 20 epochs. (This was also consistent across all eight models, but I only saved the best model when the validation loss was reduced.)
6. Reload the best model.
7. Evaluate mIoU on the first 100 validation samples.
8. Save three qualitative prediction.

7 Evaluation metric

Mean Intersection over Union is the main quantitative metric. For class c , IoU is:

$$\text{IoU}_c = \frac{TP_c}{TP_c + FP_c + FN_c}$$

where TP_c is true positives, FP_c is false positives, and FN_c is false negatives for class c . The mean IoU is the average over the 19 trainable classes:

$$\text{mIoU} = \frac{1}{19} \sum_{c=0}^{18} \text{IoU}_c$$

To avoid dividing by zero, if a class is missing from both the target and prediction for a sample, the class IoU is skipped (for example, "vegetation" or "fence" classes are not present in all images).

8 Training history summary

The training logs in text files in the output folder show that the training and validation losses generally decreased for all models. The larger models reached lower validation losses, especially the VGG16 Dilated and LRASPP models. The table below reports the final epoch losses, the best validation checkpoint, and the final mIoU evaluation.

Model	Epoch 20 train	Epoch 20 val	Best epoch	Best val	mIoU
Simple	0.8821	0.9396	19	0.9375	0.2047
Simple Dilated	0.7467	0.7963	19	0.7899	0.2331
VGG16	0.3220	0.4300	19	0.3991	0.4209
VGG16 Dilated	0.2579	0.3163	20	0.3163	0.4261
SegNet	0.4820	0.5406	20	0.5406	0.3315
SegNet Dilated	0.3961	0.4806	17	0.4511	0.3484
LRASPP	0.2149	0.3238	18	0.3110	0.3846
LRASPP Dilated	0.2380	0.3445	19	0.3403	0.3658

9 mIoU results

The final mIoU values:

Family	Base mIoU	Dilated model	Dilated mIoU	Delta
Simple	0.2047	Simple Dilated	0.2331	+0.0284
VGG16	0.4209	VGG16 Dilated	0.4261	+0.0052
SegNet	0.3315	SegNet Dilated	0.3484	+0.0169
LRASPP	0.3846	LRASPP Dilated	0.3658	−0.0188

The best model by mIoU was vgg16 dilated version, with 0.4261. Dilation improved three of the four architecture families: Simple, VGG16, and SegNet. The largest absolute improvement was for the Simple CNN family, where mIoU increased from 0.2047 to 0.2331. However, dilation did not improve LRASPP in this implementation. I can justify this by stating that the segmentation head already uses the optimal number of dilations, and that adding dilation to the encoder backbone was unnecessary and had a negative impact on final performance.

10 Inference time and number of parameters

The timing and parameter logs are available in the text files in outputs folder. Timing was measured for 10 full-resolution validation samples.

Model	mIoU	Parameters	Time for 10 samples (s)
Simple	0.2047	135,379	0.1969
Simple Dilated	0.2331	135,379	0.1970
VGG16	0.4209	8,367,379	1.0597
VGG16 Dilated	0.4261	8,367,379	1.0652
SegNet	0.3315	468,403	0.5819
SegNet Dilated	0.3484	468,403	0.6221
LRASPP	0.3846	3,221,198	0.1658
LRASPP Dilated	0.3658	3,221,198	0.1643

An important observation that I learned was that parameter count and runtime are not the same thing. LRASPP has 3.22 million parameters, much more than the Simple CNN, but it was still the fastest model in the timing test. This is because LRASPP is based on a MobileNetV3-style backbone and lightweight segmentation head. It reduces the size of high-resolution images in the network’s encoder early on. The Simple CNN has fewer parameters, but it performs dense convolutions on large feature maps, which can be expensive per pixel.

Dilation did not change the number of parameters within a model family because it changes the spacing of kernel samples, not the number of weights. Runtime changed only slightly for Simple and VGG16, increased for SegNet, and was slightly lower for LRASPP Dilated. The slight difference in timing can be explained by GPU measurement noise, and calling those layers to add dilation adds an additional overhead.

11 Qualitative Results

The project also saved visual predictions for each model. As shown in the presentation, each output PDF is divided into three panels: input image, ground truth, and model prediction.

The presentation used these examples to show that mIoU is useful but not perfect. A model can have a slightly better mIoU while still producing visually rough boundaries, or it can perform well on large classes but miss small objects.

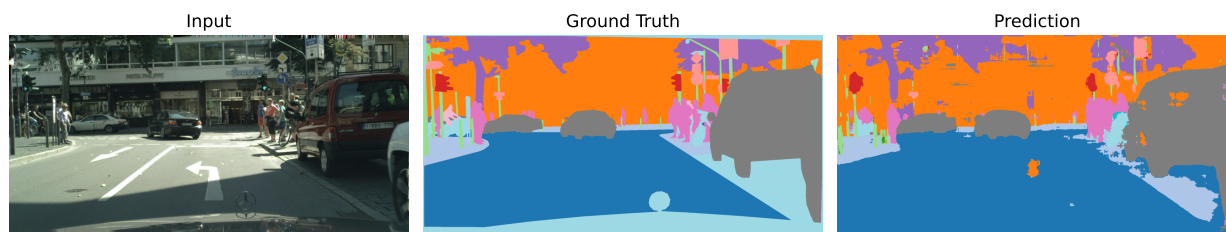


Figure 2: Qualitative output from the best mIoU model, VGG16 Dilated

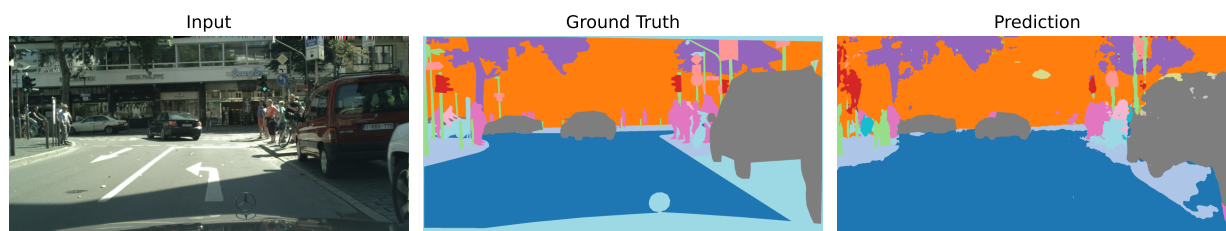


Figure 3: Qualitative output from LRASPP, which gave a strong speed and accuracy tradeoff.

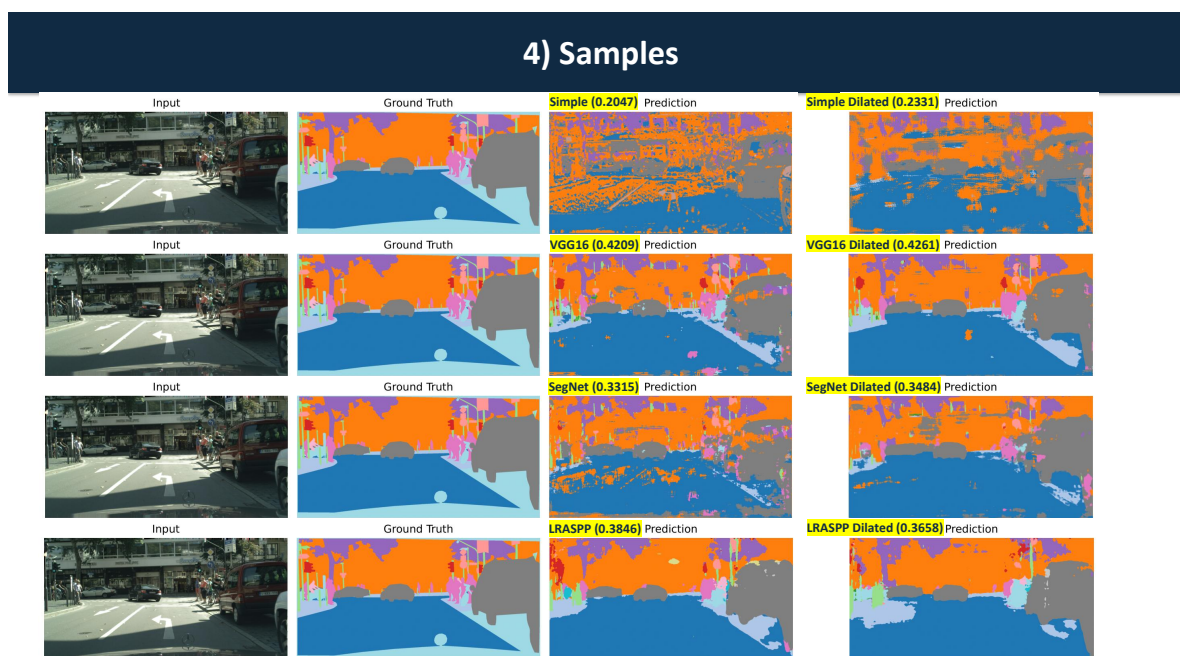


Figure 4: Sample comparison slide from the final presentation.

12 Discussion

The results show that dilation can help, but it is not automatically beneficial. In the Simple CNN, dilation increased mIoU by 0.0284. This makes sense because the model is shallow, so increasing the receptive field gives it more context. In the VGG16 family, the improvement was much smaller, only 0.0052. The VGG16-style encoder already has a larger capacity, so that might be the reason why the extra dilation helped less.

The SegNet-style model also improved with dilation, from 0.3315 to 0.3484. This suggests that the skip-concatenation decoder can still benefit from a larger receptive field. However, it became slower, increasing from 0.5819 seconds to 0.6221 seconds for 10 samples, for which I am unable to fully explain.

LRASPP was different. The base LRASPP reached 0.3846 mIoU, but the dilated LRASPP reached 0.3658. This means the dilation strategy used here was not well tuned for LRASPP. LRASPP already has an atrous-style lightweight segmentation design, and changing many backbone convolutions to dilation 2 may disturb the backbone. What I learned was this atrous-style head for dilation, and I believe that the atrous-style head is one of the best for semantic segmentation strategies one can employ. This is one of the main lessons of the project: dilation should be chosen based on architecture and feature-map resolution, not applied everywhere in the same way. It depends on the size of the input images, whether they are reduced, and the network architecture.

The strongest accuracy result came from VGG16 Dilated, but the strongest practical runtime result came from LRASPP. If the goal is highest mIoU in this project, VGG16 Dilated is the best. If the goal is speed with reasonable mIoU, LRASPP is more appealing because it is much faster than VGG16 while still achieving 0.3846 mIoU, and it can handle real-world self-driving car image segmentation much faster.

13 Limitations

Cityscapes mainly contains daytime urban driving scenes, so the models were not tested on night, heavy rain, fog, or unusual weather.

mIoU provides an average number, but it can conceal poor performance in small or safety critical classes, such as pedestrians or bikes.

14 Conclusion

This project implemented a complete semantic segmentation pipeline for Cityscapes and compared baseline and dilated versions of four architecture families. The label handling was corrected to use the official Cityscapes train-id mapping, with 19 trainable classes and 255 as the ignore index.

The main conclusion is that dilated convolutions improved three of four model families in this setup. The best final mIoU was achieved by VGG16 Dilated, with 0.4261. However, the fastest model was LRASPP Dilated, with 0.1643 seconds for 10 samples, and the best speed-accuracy compromise was the base LRASPP model, with 0.3846 mIoU and 0.1658 seconds for 10 samples.

The project therefore supports the original proposal idea that dilation can help preserve context for segmentation, but it also shows that dilation must be matched to the architecture. Applying dilation broadly is not always better.

References

- [1] Marius Cordts et al. “The Cityscapes Dataset for Semantic Urban Scene Understanding.” CVPR, 2016.
- [2] Andrew Howard et al. “Searching for MobileNetV3.” ICCV, 2019.